

Sistema multiagente para analisis financiero
historico mediante generacion y ejecucion de codigo

Nombre del alumno

2026

Resumen

Este Trabajo Fin de Master propone el desarrollo incremental de un sistema multiagente orientado al analisis financiero historico. El sistema parte de consultas ya normalizadas, datos de mercado almacenados en CSV y un conjunto cerrado de intenciones analiticas. A partir de esa entrada, el prototipo valida la solicitud, selecciona una familia de agente, genera un plan de analisis, construye codigo Python ejecutable, lanza dicho codigo en un proceso controlado y transforma la salida en una respuesta interpretable. La arquitectura se ha construido de forma gradual, manteniendo una base determinista y trazable antes de incorporar llamadas opcionales a modelos de lenguaje mediante APIs compatibles con OpenAI.

Índice general

1. Introduccion	1
1.1. Contexto y definicion del problema	1
1.2. Motivacion	1
1.3. Objetivos	2
1.4. Alcance	2
2. Estado de la cuestion	3
2.1. Sistemas basados en modelos de lenguaje	3
2.2. Arquitecturas multiagente	3
2.3. Limitaciones de los enfoques monoliticos	3
3. Material y metodos	4
3.1. Datos de entrada	4
3.2. Workflow propuesto	4
3.3. Intenciones analiticas	4
3.4. Estrategia de desarrollo incremental	5
4. Resultados y analisis	6
4.1. Cobertura funcional del MVP	6
4.2. Robustez ante errores	6
4.3. Estado de validacion	6
5. Integracion opcional de modelos de lenguaje	8
5.1. Motivacion de la integracion	8
5.2. Proveedores preparados	8
5.3. Activacion granular	8
6. Conclusiones	10
Referencias	11

A. Anexos	12
A.1. Ejecucion del MVP	12
A.2. Configuracion de APIs	12

Índice de tablas

3.1. Intenciones analíticas soportadas por el MVP.	5
4.1. Cobertura funcional validada en el MVP.	6

Índice de figuras

Capítulo 1

Introduccion

1.1. Contexto y definicion del problema

El analisis financiero historico suele combinar varias tareas que, aunque son conceptualmente distintas, aparecen juntas en el trabajo practico: carga de datos, validacion de entradas, seleccion del tipo de analisis, calculo de metricas, generacion de graficos, interpretacion de resultados y comunicacion final al usuario. En muchos prototipos basados en modelos de lenguaje, estas fases se mezclan dentro de una unica llamada generativa, lo que dificulta la trazabilidad del resultado y aumenta el riesgo de errores silenciosos.

Este trabajo plantea una aproximacion diferente. En lugar de delegar todo el proceso en un modelo generativo, se disena un flujo multiagente en el que cada fase tiene una responsabilidad delimitada. El sistema no intenta resolver desde cero el problema completo de comprension del lenguaje natural, sino que parte de una consulta financiera ya estructurada: intencion, tickers, rango temporal, intervalo y rutas a los ficheros CSV. A partir de esta entrada, el objetivo es orquestar un analisis reproducible y controlado.

1.2. Motivacion

La motivacion principal del proyecto es explorar como una arquitectura multiagente puede aportar orden, modularidad y robustez a un sistema de analisis financiero asistido por inteligencia artificial. La aplicacion de modelos de lenguaje al dominio financiero resulta atractiva por su capacidad de generar explicaciones y adaptar el lenguaje al usuario, pero tambien exige mecanismos de control. En un contexto financiero, incluso cuando el objetivo es puramente descriptivo, resulta esencial separar los calculos verificables de la redaccion final.

Por esta razon, el MVP se ha construido inicialmente con plantillas deterministas. Esta decision permite validar el flujo completo antes de introducir APIs externas o

modelos servidos localmente. Una vez estabilizada la arquitectura, la incorporacion opcional de proveedores como Groq o Gemini puede realizarse de forma gradual, empezando por la fase de interpretacion y conservando fallbacks programaticos.

1.3. Objetivos

El objetivo general del trabajo es desarrollar y validar un prototipo multiagente capaz de ejecutar analisis financieros historicos a partir de entradas estructuradas y datos en CSV.

Los objetivos especificos son:

- Definir un workflow modular que separe ingesta, enrutado, planificacion, generacion de codigo, ejecucion e interpretacion.
- Implementar familias de agentes especializadas para distintas intenciones analiticas.
- Mantener una base determinista que permita pruebas reproducibles y control de errores.
- Preparar la arquitectura para integrar modelos de lenguaje mediante APIs compatibles con OpenAI.
- Evaluar el MVP mediante casos funcionales y escenarios de error controlado.

1.4. Alcance

El sistema se limita al analisis historico-descriptivo de activos financieros a partir de datos ya descargados. No realiza prediccion futura, recomendacion de inversion ni analisis fundamental complejo. Esta restriccion es deliberada: el foco del TFM no es construir un asesor financiero, sino estudiar la arquitectura de un pipeline multiagente trazable para analisis historico.

Capítulo 2

Estado de la cuestion

2.1. Sistemas basados en modelos de lenguaje

Los modelos de lenguaje de gran escala han abierto la posibilidad de construir sistemas capaces de interpretar instrucciones, generar código y redactar explicaciones adaptadas al usuario. Sin embargo, su uso directo en tareas analíticas puede introducir problemas de verificabilidad si no se acompaña de estructuras de control. En dominios donde los resultados dependen de cálculos concretos, como el análisis financiero, resulta conveniente combinar componentes generativos con ejecución programática y validaciones explícitas.

2.2. Arquitecturas multiagente

Las arquitecturas multiagente permiten dividir una tarea compleja en responsabilidades más pequeñas. En este proyecto, la idea de agente no se entiende como una entidad autónoma sin restricciones, sino como una familia analítica con una interfaz clara: recibe una intención, produce un plan, genera o selecciona código y contribuye a la interpretación final. Esta separación facilita la extensión del sistema, ya que nuevas intenciones pueden incorporarse sin reescribir el flujo completo.

2.3. Limitaciones de los enfoques monolíticos

Un enfoque monolítico, en el que una única función o prompt realiza todo el proceso, puede ser suficiente para una demostración pequeña, pero presenta limitaciones cuando se desea evaluar el sistema de forma rigurosa. Entre ellas destacan la dificultad para aislar errores, la falta de trazabilidad entre la entrada y la salida, y la ausencia de puntos intermedios donde aplicar validaciones. El diseño propuesto en este TFM busca precisamente evitar estas limitaciones.

Capítulo 3

Material y metodos

3.1. Datos de entrada

El MVP trabaja con datos financieros historicos almacenados en ficheros CSV. Cada ejecucion recibe una consulta ya estructurada con los siguientes campos principales: texto original de la consulta, intencion analitica, lista de tickers, periodo o fechas de inicio y fin, intervalo temporal y rutas a los CSV correspondientes.

3.2. Workflow propuesto

El flujo del sistema se organiza en seis fases principales:

1. **Ingesta:** valida la entrada minima y comprueba que los ficheros existen.
2. **Enrutado:** decide si la intencion esta soportada y selecciona la familia analitica.
3. **Analisis especializado:** transforma la intencion en un plan estructurado.
4. **Generacion de codigo:** construye un script Python adaptado al plan.
5. **Ejecucion:** ejecuta el codigo mediante un proceso controlado y captura salidas.
6. **Interpretacion:** convierte los resultados numericos en una respuesta legible.

3.3. Intenciones analiticas

La version actual del MVP soporta seis intenciones:

Intencion	Objetivo
price_growth	Calcular crecimiento o caida de un activo en un periodo.
compare_assets	Comparar la evolucion relativa de varios activos.
asset_overview	Generar una vision descriptiva general de un activo.
return_analysis	Analizar retornos historicos y estadisticos basicos.
historical_risk_analysis	Estimar riesgo historico mediante volatilidad y draw-down.
technical_analysis	Calcular indicadores tecnicos como medias moviles y RSI.

Tabla 3.1: Intenciones analiticas soportadas por el MVP.

3.4. Estrategia de desarrollo incremental

El desarrollo se ha organizado por iteraciones. Primero se valido un flujo minimo con pocas intenciones. Despues se ampliaron las capacidades descriptivas, de retornos y riesgo. Finalmente se incorporo analisis tecnico, mayor cobertura de tests y una modularizacion por familias de agentes. Esta estrategia reduce el riesgo tecnico antes de incorporar modelos de lenguaje reales.

Capítulo 4

Resultados y analisis

4.1. Cobertura funcional del MVP

El resultado principal hasta este punto es un MVP capaz de completar el flujo de analisis para seis intenciones financieras. La arquitectura mantiene una separacion clara entre validacion, seleccion de agente, generacion de plan, generacion de codigo, ejecucion e interpretacion.

Intencion	Caso representativo	Estado
price_growth	Crecimiento de NVDA	Superado
compare_assets	Comparacion NVDA y AMD	Superado
asset_overview	Vision general de AAPL	Superado
return_analysis	Retornos de QQQ y SPY	Superado
historical_risk_analysis	Riesgo historico de QQQ y SPY	Superado
technical_analysis	Analisis tecnico de AAPL	Superado

Tabla 4.1: Cobertura funcional validada en el MVP.

4.2. Robustez ante errores

Ademas de los casos correctos, el MVP contempla escenarios negativos. Entre ellos se incluyen intenciones no soportadas, rutas CSV inexistentes y cardinalidad incorrecta de tickers para analisis que requieren una unica serie. En estos casos, el sistema no interrumpe el flujo de forma opaca, sino que devuelve un estado de error controlado y una explicacion para el usuario.

4.3. Estado de validacion

La bateria actual de pruebas automatizadas valida tanto ejecuciones correctas como errores esperados. En la ultima ejecucion realizada, los tests del proyecto finalizaron

correctamente, lo que confirma que la integracion opcional de APIs LLM no rompe el modo determinista.

Capítulo 5

Integracion opcional de modelos de lenguaje

5.1. Motivacion de la integracion

Una vez consolidado el MVP determinista, el siguiente paso consiste en incorporar modelos de lenguaje de forma controlada. La arquitectura no depende de un proveedor concreto: se ha preparado un cliente compatible con APIs tipo OpenAI, lo que permite alternar entre distintos backends sin modificar el workflow principal.

5.2. Proveedores preparados

En esta fase se han preparado dos perfiles de API:

- **Groq**: seleccionado como primera opcion por su compatibilidad con OpenAI, baja latencia y facilidad de integracion.
- **Gemini**: incorporado como segunda opcion para comparar calidad y robustez de las respuestas.

5.3. Activacion granular

La integracion LLM se ha disenado para activarse por fases:

- **LLM_USE_FOR_INTERPRETATION**: el modelo redacta la respuesta final a partir de resultados ya calculados.
- **LLM_USE_FOR_PLANNING**: el modelo puede ajustar el plan analitico sin cambiar la intencion.

- `LLM_USE_FOR_CODEGEN`: el modelo puede generar código Python bajo un contrato mínimo.

La recomendación actual es comenzar solo por la interpretación, ya que es la fase con menor riesgo técnico. El cálculo de métricas permanece controlado por código determinista y, si la API falla o la respuesta no cumple el contrato esperado, el sistema conserva el fallback programático.

Capítulo 6

Conclusiones

El trabajo desarrollado hasta este punto demuestra la viabilidad de un sistema multiagente para analisis financiero historico basado en una arquitectura modular y trazable. La decision de comenzar con una base determinista ha permitido validar el flujo completo antes de introducir componentes generativos, reduciendo el riesgo de errores dificiles de diagnosticar.

El MVP actual soporta seis intenciones analiticas, gestiona errores basicos de entrada y separa las responsabilidades en familias de agentes. Esta estructura facilita la evolucion del sistema hacia una version hibrida, donde los modelos de lenguaje puedan aportar flexibilidad en la planificacion, generacion de codigo o interpretacion final sin sustituir los mecanismos de validacion y ejecucion controlada.

Como trabajo futuro, se plantea ampliar la evaluacion experimental, comparar el comportamiento de Groq y Gemini en la fase de interpretacion, estudiar la activacion progresiva del LLM en planificacion y analizar bajo que condiciones resulta seguro incorporar generacion de codigo asistida por modelo.

Referencias

- [1] LangChain, «LangGraph: Building Stateful, Multi-Actor Applications with LLMs,» 2024. dirección: <https://langchain-ai.github.io/langgraph/>
- [2] Groq, *Groq API Documentation*, 2026. dirección: <https://console.groq.com/docs>
- [3] Google AI for Developers, *Gemini API Documentation*, 2026. dirección: <https://ai.google.dev/gemini-api/docs>
- [4] R. Aroussi, *yfinance Python Library*, 2026. dirección: <https://github.com/ranaroussi/yfinance>

Apéndice A

Anexos

A.1. Ejecucion del MVP

El proyecto puede ejecutarse en modo determinista mediante los ejemplos incluidos en el repositorio:

```
python run_mvp.py --example growth_nvda
python run_mvp.py --example compare_nvda_amd
python run_mvp.py --example overview_aapl
python run_mvp.py --example returns_qqq_spy
python run_mvp.py --example risk_qqq_spy
python run_mvp.py --example technical_aapl
```

A.2. Configuracion de APIs

Las claves de API se guardan localmente en el fichero `.env`, que no debe subirse al repositorio. Para trabajar sin APIs externas:

```
LLM_ENABLED=false
```

Para activar Groq:

```
LLM_ENABLED=true
LLM_PROFILE=groq
```

Para activar Gemini:

```
LLM_ENABLED=true
LLM_PROFILE=gemini
```